

React Interview Questions:

EASY (BEGINNER LEVEL) :

- What is react and why is it popular?
 - **React** is a **JavaScript library** for building **user interfaces (UIs)**, particularly for single-page applications (SPAs). It was developed by **Facebook (now Meta)** and is maintained by Facebook along with a large open-source community.
 - React allows developers to build **reusable UI components** that update efficiently in response to data changes. Instead of manipulating the **DOM (Document Object Model)** directly, React uses a concept called the **Virtual DOM** to optimize rendering performance.
- > Here are the key reasons for its popularity:
- **Component-Based Architecture :**
 - ◆ React apps are built using components. A component is an independent, reusable piece of UI. Components can have their own state and logic. This makes it easy to understand, test and reuse across different parts of the application.
 - **Declarative Syntax :**
 - ◆ In programming, **declarative syntax** means **describing what you want to happen**, rather than explicitly coding how to make it happen step by step.
 - ◆ You describe *what* you want the UI to look like, and React takes care of the rest.
 - ◆ Easier to debug and reason about.
 - **Virtual DOM for Performance :**
 - ◆ The **Virtual DOM** is an in-memory representation of the real DOM elements.
 - ◆ It is a **lightweight JavaScript object** that mirrors the structure of the actual DOM.
 - ◆ React uses it to figure out **what has changed** before touching the real DOM, which is slow to update directly.
 - ◆ How React Uses the Virtual DOM:
 - ◇ **Render Phase:** React renders the entire UI into the Virtual DOM when a component's state or props change.
 - ◇ **Diffing:** React compares (or "*diffs*") the new Virtual DOM with the previous one to identify what changed.
 - ◇ **Reconciliation:** React calculates the **minimum number of**

operations needed to update the real DOM based on the differences.

- ◊ **Update the real DOM:** Only the **changed parts** of the DOM are updated, not the entire DOM.
- ◆ Why Use the Virtual DOM?
 - ◊ **Performance:** Real DOM manipulations are expensive; VDOM makes updates more efficient.
 - ◊ **Declarative UI:** You declare *what* the UI should look like; React handles the *how* via VDOM.
 - ◊ **Consistency:** Ensures UI stays in sync with the app's state.
- **Strong Ecosystem and Community :**
 - ◆ **Rich ecosystem:** React Router, Redux, Next.js, etc.
 - ◆ **Large community:** Tons of libraries, tutorials, and support.
- **Used by Big Companies :**
 - ◆ Facebook, Instagram, Airbnb, Netflix, Uber, and many more.
 - ◆ Real-world use validates its performance and scalability.
- **JSX – JavaScript + XML :**
 - ◆ JSX is a syntax extension for javascript that allows you to write HTML like code in javascript files. React uses JSX to define components and visualize the UI structure.
- **One-way data binding :**
 - ◆ In react, data flows in only one direction i.e from parent to child. This helps in understanding how data affects the application and helps prevent bugs that can arise from two-way binding.
- **React Hooks :**
 - ◆ **Hooks** are functions that let you "hook into" React state and lifecycle features from function components.
 - ◆ Hooks like `useState` (for managing state) and `useEffect` (for handling side effects) allow developers to use state and other React features in a more concise and functional way, leading to cleaner and more readable code.
- **React Native for Mobile Apps :**
 - ◆ Same React principles can be used to build **native mobile apps** for iOS and Android.

– How do you create a React component?

- In React, components can be created in two ways: **Functional**

Components and Class Components.

- **Functional Components** are JavaScript functions that accept props and return JSX. They can manage state and side effects using **Hooks** like `useState` and `useEffect`, a capability that wasn't possible before React 16.8. These components are simpler and more concise.

```
◆ function Counter() {
◆   const [count, setCount] = React.useState(0);
◆   return (
◆     <div>
◆       <p>Count: {count}</p>
◆       <button onClick={() => setCount(count + 1)}>Increment</
button>
◆     </div>
◆   );
◆ }
```

- **Class Components** are ES6 classes that extend `React.Component`. They manage state using `this.state` and `this.setState`. They also have access to **lifecycle methods** such as `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`. They involve more boilerplate and can be harder to reason about compared to functional components

```
◆ class MyComponent extends React.Component {
◆   constructor(props) {
◆     super(props); // Must call super before accessing this
◆     this.state = { count: 0 };
◆   }
◆
◆   increment = () => {
◆     this.setState({ count: this.state.count + 1 });
◆   }
◆
◆   render() {
◆     return (
◆       <div>
◆         <p>Count: {this.state.count}</p>
◆         <button onClick={this.increment}>Increment</button>
◆       </div>
◆     );
◆   }
◆ }
```

- What is the purpose of the render method in a React Component?
- The purpose of the render() method in a React component is to describe what the UI should look like.
 - It **returns JSX** (or null), which tells React **what to display** on the screen.
 - React uses the output of the render() method to **build and update the DOM**.
 - It is **required** in class components (but not used in functional components).
- What is the significance of the state in React?
- The **state** in React is a **built-in object** that allows components to **store and manage dynamic data** that can **change over time** and influence what is rendered on the screen.

Significance of State in React:

- **Dynamic UI Updates**
State allows components to react to user input, API responses, or internal changes by automatically re-rendering the UI when the state changes.
- **Component-Specific Data**
State is local to a component and helps it track data that shouldn't be managed globally (like input field values, toggle status, counters, etc.).
- **Triggers Re-rendering**
When the state changes using functions like setState (class components) or useState (functional components), React re-renders the component to reflect the updated data.
- **Drives Interactivity**
Without state, React components would be mostly static. State enables features like:
 - ◆ Form inputs
 - ◆ Toggling modals
 - ◆ Tab switching
 - ◆ Real-time updates

- How do you pass data from a parent component to its child component?
- You pass data from a parent component to a child component in React using props (short for "properties").

How It Works:

- In the **parent component**, you provide data as an attribute on the child component.
- In the **child component**, you access that data via the props object.
 - ◆ `// ParentComponent.js`
 - ◆ `function ParentComponent() {`
 - ◆ `const message = "Hello from Parent!";`
 - ◆
 - ◆ `return <ChildComponent text={message} />;`
 - ◆ `}`
 - ◆
 - ◆ `// ChildComponent.js`
 - ◆ `function ChildComponent(props) {`
 - ◆ `return <p>{props.text}</p>;`
 - ◆ `}`

Key Points about Props:

- **Read-only:** Props are immutable inside the child component. The child cannot change the props it receives.
 - **Passed from parent:** They enable **one-way data flow** from parent to child components.
 - **Used to customize components:** Props help make components **reusable** and **dynamic** by allowing you to pass different data or configuration.
 - **Accessed via props object:** In functional components, props are received as a function argument; in class components, via `this.props`.
- What is React's key principle for optimizing performance?

- React's key principle for optimizing performance is efficiently updating the UI by minimizing direct DOM manipulations through a virtual DOM and using reconciliation to update only what's necessary.
 - **Virtual DOM**: React keeps a lightweight copy of the real DOM in memory (the virtual DOM). When the state or props change, React updates the virtual DOM first.
 - **Reconciliation**: React compares the new virtual DOM with the previous one (called *diffing*). It figures out the minimal set of changes needed.
 - **Batch Updates**: React batches multiple changes to avoid unnecessary re-renders.
 - **Selective Rendering**: By using keys and techniques like `shouldComponentUpdate`, `React.memo`, and hooks like `useMemo`, React avoids re-rendering components whose data hasn't changed.
- ◆ The main **optimization techniques** React provides for **selective rendering**, so components **only re-render when needed**, improving performance:
 - ◇ **`shouldComponentUpdate()` (Class Components)** : A lifecycle method in class components that **controls whether a component should re-render** when props or state change.


```

          ■ class MyComponent extends React.Component {
          ■   shouldComponentUpdate(nextProps, nextState) {
          ■     // Return false to skip re-rendering
          ■     return nextProps.value !== this.props.value;
          ■   }
          ■
          ■   render() {
          ■     return <p>{this.props.value}</p>;
          ■   }
          ■ }
          
```

Use Case:

 - Prevent re-renders if the data hasn't changed.
 - Helps avoid unnecessary DOM updates.
 - ◇ **`React.memo()` (Functional Components)** : A higher-order component that **wraps functional components** to prevent them from re-rendering unless their **props actually change**.

- `const MyComponent = React.memo(function MyComponent(props) {`
- `return <p>{props.value}</p>;`
- `});`

With custom comparison:

- `const MyComponent = React.memo(`
- `function MyComponent(props) {`
- `return <p>{props.value}</p>;`
- `},`
- `(prevProps, nextProps) => {`
- `return prevProps.value === nextProps.value; // true = skip render`
- `}`
- `);`

Use Case:

- Ideal for **pure** components (those that re-render only when their props change).

- ◇ **useMemo() (Functional Components)** : A React Hook that **memoizes expensive calculations** and **returns a cached value** unless dependencies change.

- `const expensiveResult = useMemo(() => {`
- `return heavyComputation(input);`
- `}, [input]);`

Use Case:

- Useful when **calculating derived data** that doesn't need to be recalculated on every render.

- ◇ **useCallback() (Functional Components)** : A hook that **memoizes functions**, preventing re-creation on every render unless dependencies change.

- `const handleClick = useCallback(() => {`
- `console.log("Clicked");`
- `}, []);`

Use Case:

- Prevents **unnecessary re-renders** of child components receiving callback props.

- Which component, functional or class, is commonly used now and why?
- Functional components are more commonly used today because of the following reasons:
 - They are simple and concise. Its much easier to read and understand the code compare to the class components.
 - The hooks namely `useState` and `useEffect` have simplified state management and side-effects.
 - React team has stated the future performance optimizations will likely focus on functional components.
 - Easier to test.
- Can you name a few common lifecycle methods in class components and describe when they are called. Also, how do you achieve similar lifecycle behaviour in functional components using hooks?
- Following are the most commonly used lifecycle methods in React class components:
 - **`constructor()`**: This is the very first method that gets called when a component is being created and initialized. it is primarily used to setup initial state of the component and bind event handler methods.
 - **`render()`**: This is a required method in every class component. React uses `render` to figure what needs to be displayed on the screen. It is responsible for returning the JSX that describes the UI.
 - **`componentDidMount()`**: This method is invoked immediately after a component is mounted. Its a common place to perform tasks that require DOM node or to fetch data from an API..
 - **`componentDidUpdate()`**: This method is invoked immediately after a components updates are flushes to the DOM. It's useful for the actions that depend on updated DOM or for re-fetching data if certain props or state has changed.
 - **`componentWillUnmount()`**: This method is invoked immediately before a component is unmounted and destroyed. It is a place for clean up tasks such as invalidate timers, remove event listeners if setup before, cancel network requests etc.

- You achieve similar lifecycle behaviour in functional components using `useEffect` hook. `useEffect` hook basically combines the functionality of `componentDidMount()`, `componentDidUpdate()` and `componentWillUnmount()` into a single API. It takes two arguments:
 - A callback function containing the side effect logic.
 - An optional dependency array which tells react when to rerun the effect.
- Can you explain how you implement cleanup in `useEffect`? For example, how would you remove an event listener or cancel a timer inside a functional component?
- Give a quick code snippet if you'd like.
 - ```
function Timer() {
 const [seconds, setSeconds] = useState(0);
 useEffect(() => {
 const intervalId = setInterval(() => {
 setSeconds(prevSeconds => prevSeconds + 1);
 }, 1000);
 return () => {
 clearInterval(intervalId);
 }
 }, []);
 return (
 <div>
 Time elapsed: {seconds}s
 </div>
);
}
```
- What happens if you forget to include the **dependency array** in `useEffect`?
- Also, what happens if you omit the **cleanup function** when using things like `setInterval` or event listeners?
- The **dependency array** scenarios:
  - **No Dependency Array** (`useEffect(() => { ... })`) :

- ◆ Runs **after every render**, including:
  - ◇ Initial render
  - ◇ Any re-render (even if it's caused by unrelated state/prop changes)
- ◆ **Problem:** Can cause performance issues and unwanted side effects (e.g. multiple event listeners, intervals piling up)
- **Empty Dependency Array** (`useEffect(() => { ... }, [])`) :
  - ◆ Runs only **once** on mount (like `componentDidMount`)
  - ◆ Cleanup (if provided) runs on unmount (like `componentWillUnmount`)
- **Specific Dependencies** (`useEffect(() => { ... }, [someState])`) :
  - ◆ Runs only when **someState** changes
  - ◆ Cleanup (if provided) runs **before the next effect execution** and on unmount
- If you **don't return a cleanup function** from `useEffect`, then things like intervals, event listeners, or subscriptions will **persist** even after the component unmounts or re-renders. This can lead to:
  - **Memory leaks**
  - **Duplicate intervals/listeners**
  - **Unexpected bugs** like triggering logic multiple times

## MEDIUM ( INTERMEDIATE / MID-LEVEL) :

### – What is Virtual DOM?

- The Virtual DOM is an in-memory representation of the real DOM. When the state of a component changes in frameworks like React, a new Virtual DOM is created. React then compares this new Virtual DOM with the previous one using a process called "diffing." After identifying what has changed, React updates only those specific parts in the actual DOM.
- This makes updates more efficient, since manipulating the real DOM directly is relatively expensive in terms of performance.

- **What makes the Virtual DOM faster**, given that there's still some work being done during diffing?
  - Although diffing also takes computation, it's typically much faster because it's done in memory — not directly on the real DOM, which involves layout calculations, reflows, and repaints.
  - Moreover, frameworks like React optimize this process further using techniques like **reconciliation**, **component shouldComponentUpdate logic**, or React's **memoization** with `React.memo` and `useMemo`, which prevents unnecessary re-renders.
- Can you explain the **React component lifecycle**, and how it differs between class components and functional components?
  - In **class components**, the lifecycle is broken down into phases:
    - ◆ **Mounting**: `constructor`, `componentDidMount`
    - ◆ **Updating**: `shouldComponentUpdate`, `componentDidUpdate`
    - ◆ **Unmounting**: `componentWillUnmount`
  - In **functional components**, React introduced Hooks to manage lifecycles using `useEffect`. The `useEffect` hook can replicate:
    - ◆ `componentDidMount` → `useEffect(() => { ... }, [])`
    - ◆ `componentDidUpdate` → `useEffect(() => { ... }, [deps])`
    - ◆ `componentWillUnmount` → using a return cleanup function inside `useEffect`
  - Functional components provide a cleaner and more concise way to manage side effects and lifecycles.
- **What are some potential pitfalls** developers run into when using `useEffect`?
  - Some common pitfalls include:
    - ◆ **Missing dependencies**: Not including all necessary dependencies in the dependency array can lead to stale closures or unexpected behavior.

- ◆ **Over-triggering effects:** Including unstable values like inline functions or objects in the dependency array can cause the effect to run on every render.
- ◆ **Memory leaks:** Forgetting to clean up subscriptions, timers, or event listeners inside the cleanup function can cause performance issues.

Using **useCallback**, **useMemo**, and refactoring logic outside of components can help mitigate these.

– What is **prop drilling**, and how can it be avoided?

- **Prop drilling** happens when you pass props through multiple levels of components just to reach a deeply nested child. This can make components tightly coupled and harder to maintain.
- To avoid it, we can use:
  - ◆ **React Context API:** Share state or functions globally within a component tree.
  - ◆ **State management libraries** like Redux, Zustand, or Recoil, for larger apps.
  - ◆ **Component composition** or **restructuring** to reduce deeply nested trees.
- If you're using **React Context**, what's one performance issue it might introduce?
  - Context re-renders all consuming components whenever its **value changes**, even if only part of the value is updated. This can cause unnecessary renders.
  - To avoid that, we can :
    - ◆ **Split context** into multiple smaller contexts.
    - ◆ Use **useMemo** to **memoize** the context value.
    - ◆ In complex cases, I might use **libraries like use-context-selector** to optimize granular subscriptions.

– How does **React's reconciliation** algorithm work?

- React's reconciliation algorithm is how React updates the DOM when a component's state or props change.
- It compares the new virtual DOM with the previous one using a **diffing algorithm**, and only updates the parts of the DOM that have changed.
- Some Key Points:
  - ◆ React assumes elements of **different types** produce different trees.
  - ◆ React uses **keys** to match children in lists for efficient updates.
  - ◆ Components are **reused** if their keys and types match.

● Why is using **keys** important in lists, and what happens if you use the index as a key?

- Keys help React identify which items have changed, been added, or removed. When you use **indexes as keys**, it can lead to incorrect DOM updates, especially during insertions or deletions, because the index might not uniquely identify the item.
- This can break component state or cause unwanted re-renders. It's better to use a **stable and unique identifier**, like an id.

– What is the difference between **useMemo** and **useCallback**?

- **useMemo** is used to **memoize a computed value** so it's only recalculated when its dependencies change.
- **useCallback** memoizes a **function**, so a new function reference isn't created on every render.
- Both help with performance, especially when passing values or functions to child components that rely on **React.memo**.
  - ◆ `const expensiveValue = useMemo(() => computeHeavy(), [input]);`

- ◆ `const memoizedFn = useCallback(() => doSomething(id), [id]);`
- The situations would you *not* use `useMemo` or `useCallback` :
  - ◆ Avoid using them **prematurely**. If the computation is cheap or the function isn't being passed to memoized children, then memoization adds unnecessary complexity and overhead.
  - ◆ React itself is pretty fast. It's better to profile first using React DevTools or performance tools, and only introduce memoization where it truly improves performance.
- What happens when you call `setState` in React? Is it synchronous or asynchronous?
  - Calling `setState` in React schedules a re-render.
  - In **class components**, it's always **asynchronous**. In **functional components**, `setState` (from `useState`) is **asynchronous within React's event handlers**, but might behave **synchronously** in non-React contexts like `setTimeout`.
  - React **batches** multiple `setState` calls together to optimize rendering.
- How can you update state based on the previous state?
  - You should pass a function to `setState` or `setX`:
    - ◆ `setCount(prevCount => prevCount + 1);`
  - This ensures you're working with the latest state, especially when multiple updates are batched together.
  - If you call `setState` multiple times in the same event:
    - ◆ `setCount(count + 1);`
    - ◆ `setCount(count + 1);`
  - You might think the count will go up by 2 — but actually, it will go up by just **1** unless you use the **functional form**:
    - ◆ `setCount(prev => prev + 1);`
    - ◆ `setCount(prev => prev + 1);`

- This will increase the count by **2**, because it uses the most recent value.

- SUMMARY:

| Question                             | Answer                                                                                                                                               |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Is setState synchronous?             |  No — it's <b>asynchronous in behavior</b> (batched and deferred)   |
| Does it update the state right away? |  No — the update happens after React re-renders                     |
| How to get the new state?            |  Use useEffect, or use the updated value in the next render         |
| How to ensure multiple updates work? |  Use functional updates: <code>setCount(prev =&gt; prev + 1)</code> |

- What are controlled vs uncontrolled components in React forms?

- **Controlled components** are form elements (like inputs) where React **controls the value** via state and updates it via onChange.
  - ◆ `const [value, setValue] = useState('');`
  - ◆ `<input value={value} onChange={e => setValue(e.target.value)} />`
- **Uncontrolled components** use **refs** to access the DOM element's current value directly, not managed by React.
  - ◆ `const inputRef = useRef();`
  - ◆ `<input ref={inputRef} />`
- Controlled components are more predictable and integrate better with form validation and React state.
- You might use uncontrolled components when:
  - ◆ You need **quick form inputs** without tight control (e.g., file uploads).
  - ◆ You're integrating with a **third-party library** that manages its own DOM.
  - ◆ You want **less re-renders** for large forms where state control causes lag.

- ◆ But in most cases, especially with validation or complex logic, You use controlled components.

– Explain how React Keys work internally?

- React uses **keys** to identify which elements in a list have changed. During reconciliation, it compares the keys of the old and new virtual DOM to decide:
  - ◆ If the element should be **reused**, **reordered**, or **removed**.
  - ◆ Keys must be **stable and unique** among siblings.
- Internally, React builds a map of keys and compares them to avoid unnecessary unmounting and remounting.
- If two elements share the same key, i.e Using duplicate keys can confuse React's diffing algorithm. It might:
  - ◆ Reuse the wrong component.
  - ◆ Cause weird UI bugs.
  - ◆ Lead to performance issues.
  - ◆ React throws a warning in dev mode to help catch this.